

ENTERPRISE AI OPERATING MODEL · PROOF OF CONCEPT KICKOFF

Standing up your agent fabric

One operating model. Six planes you mostly already own. Twelve weeks to a paved road that **[CLIENT]**'s teams run themselves — starting with agents that build and review your own software.

Ninety minutes. One canvas, one offer, one plan.

You will not need to take notes. Everything here is in the leave-behind, and the deep technical material sits in the appendix — say the word and we will go there.

TIME	WHAT WE COVER	WHO THIS IS FOR
0:00	The operating model on one canvas — and how it unpacks	Everyone
0:18	What we deliver, what it costs, what it returns	Executives · sponsors
0:34	How the fabric works — event plane, safe autonomy, knowledge, agentic delivery	Architecture · platform · AI practice
0:56	Where each role works — and a day in the life afterwards	All disciplines
1:12	Maturity path and improvement roadmap	Everyone
1:26	Plan, dependencies, decisions	Sponsor

One ask on interruptions: if we describe your environment wrongly, stop us immediately. We would rather be corrected in minute twelve than build on it for a month.

WE NEED FROM YOU

Five decisions

- 1 Confirm the **first squad** and its product owner
- 2 Confirm **agentic delivery** as the first workload
- 3 Confirm the **incumbent platforms** we build on
- 4 Name the **seven roles** and the weekly rhythm
- 5 Agree the **value measures** we will report against

None of these require a business case. They require a room, which you already have.

Any team can build one agent. Nobody can build the hundredth without a fabric.

The models are ready. What is missing is the shared substrate that makes the second, tenth and hundredth agent cheap, observable and safe to run — a registry, a way to grant authority, one event spine, a knowledge layer and a paved road to production. Built once, used by everyone.

TODAY

Every agent is a project

Bespoke integration, bespoke review, bespoke ops. Cost per capability stays flat, so the portfolio never compounds.

WITH A FABRIC

Every agent is a deployment

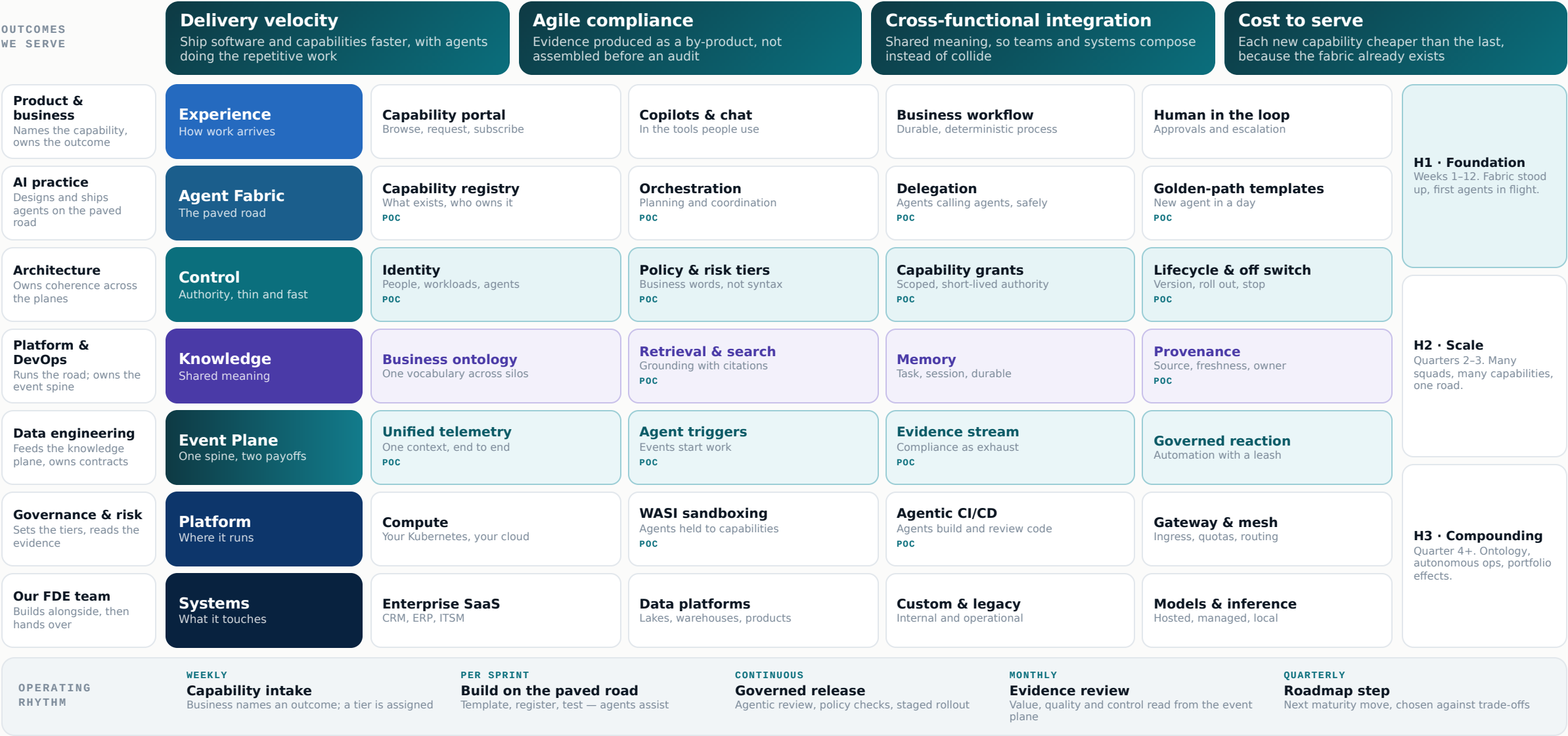
Register a capability, inherit identity, policy, telemetry and a release path. Cost per capability falls with every one you add.

THE COMPOUNDING

Capability #10 costs a fraction of #1

Because #1 paid for the fabric. This is the entire economic argument for doing platform work before portfolio work.

The whole model on one page — click any block to unpack it.



Four things leave our hands and stay in yours.

Twelve weeks, one squad, working software. Everything below is either running in your estate or checked into your repositories when we finish.

DELIVERABLE 01

A working agent fabric

Capability registry, orchestration, delegation, identity and policy — deployed in your environment, on your platform.

You can: register a new agent capability and see it appear with an owner, a tier, health and an off switch.

DELIVERABLE 02

Agents doing real work

Agentic CI/CD: code review, test generation, dependency upgrades, release notes and deploy gating — on your repositories.

You can: open a pull request and get a substantive agent review before a human reads it.

DELIVERABLE 03

The paved road

Templates, registration, policy inheritance, test harness, dashboards and runbooks — so the next squad does not need us.

You can: stand up a new agent capability in about a day, from documentation alone.

DELIVERABLE 04

The operating rhythm

Capability intake, tiering, release and evidence review — with the forums, owners and measures defined and running.

You can: answer "what are our agents doing, and is it working?" from one place, monthly.

Why delivery agents first? Internal, reversible, no customer exposure, and measurable inside three weeks — while proving every plane of the fabric under real load. Business capabilities are path two, designed in week ten with everything we learned.

What it costs, and the shape of what it returns.

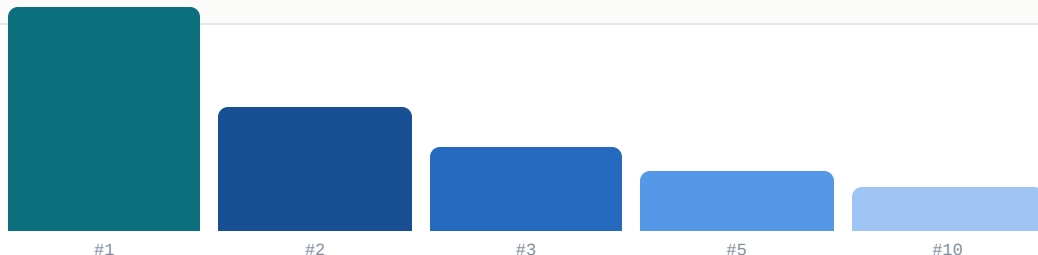
Bracketed figures are modelled with your data in week one. We will not put invented numbers in front of you — but the shape of the case does not depend on the numbers, and the shape is the argument.

THE INVESTMENT • TWELVE WEEKS

Our delivery squad — 5 forward-deployed engineers	12 wks
Your team — seven named roles, part time	~4 d/wk
Infrastructure — your existing platform	no new spend
Incremental run cost — inference and control footprint	[\$X] /mo
Licence commitments required to start	none

Nothing here is a platform purchase. The fabric is built from what you already run, plus components we hand over as your code.

COST PER [N] CAPABILITY — WHY THE FIRST ONE LOOKS EXPENSIVE



Illustrative shape, not a forecast. Capability #1 carries the fabric; #10 carries only itself. **Pricing agent number one as a product decision will always say no.**

THE RETURN • FOUR DRIVERS, MEASURED FROM THE EVENT PLANE

Engineering throughput

VISIBLE WK 3

Review latency, rework rate and change lead time on the squads running agentic CI/CD, against their own pre-POC baseline.

DRIVER • [X] % cycle-time reduction × [N] engineers

Compliance effort

VISIBLE WK 9

Evidence is emitted continuously rather than assembled per audit. The saving is analyst days per control cycle, not software licences.

DRIVER • [N] analyst-days × [N] cycles/yr

Integration cost avoided

VISIBLE Q2

Shared capability contracts and vocabulary replace point-to-point work between teams and systems.

DRIVER • [N] integrations × [\$X] avg build

Portfolio compounding

COMPOUNDS

The dominant term after the first two quarters, and the reason this is a platform decision rather than a project decision.

DRIVER • marginal cost of capability n → [X] % of capability #1

You already own two-thirds of the fabric. We are adding a thin layer.

Our starting read, to be corrected by you in the next five minutes. The middle column is blank on purpose — we fill it together, live.

WHAT THE FABRIC NEEDS	[CLIENT] ALREADY HAS	EFFORT
Compute & orchestration	—	● REUSE
Identity provider	—	● REUSE
CI/CD pipelines	—	● REUSE
Observability backend	—	● REUSE
API gateway / mesh	—	● REUSE

WHAT THE FABRIC NEEDS	[CLIENT] ALREADY HAS	EFFORT
Messaging / event bus	—	● REUSE
Data platform & catalog	—	● REUSE
Telemetry conventions	—	● EXTEND
Policy & risk tiers	—	● EXTEND
Capability registry	—	NEW · WE BUILD
Capability grants	—	NEW · WE BUILD

● **Reuse** — we integrate with what you run today

● **Extend** — a convention or contract added to something existing

● **New** — built by us in the twelve weeks, owned by you after

SIX.

One event spine. Three payoffs from a single investment.

Most enterprises fund observability and eventing separately, out of different budgets, and build the same spine twice. Unify it once and agents become possible almost as a side effect.

THE SINGLE INVESTMENT

A unified event plane

One context propagated through every call. One bus. One schema for what happened, who asked, what was decided and what changed.

Built on what you run. Your existing bus, your existing observability backend. We add conventions, not infrastructure.

IN POC SCOPE

PAYOFF 01

Extensible observability

Every agent, service and decision on one correlated trace. New capabilities inherit instrumentation instead of adding a monitoring backlog item.

Who wins: platform, SRE, and every team that would otherwise be told to "add logging".

PAYOFF 02

Agent triggers

Work starts because something happened — a PR opened, a ticket raised, a threshold crossed — not because a human remembered to ask.

Who wins: AI practice and the business. **An agent that can only be called is a tool; one that can be triggered is a colleague.**

PAYOFF 03

Evidence as exhaust

The record auditors want is produced by the running system, continuously, rather than assembled by analysts before a review.

Who wins: governance, risk and audit — see the maturity path on slide 18.

The honest trade-off: one spine means one thing to get right. It needs schema discipline, back-pressure handling and an owner. We would rather you run one spine well than three badly — but it is a real commitment, and the platform team should hear that from us rather than discover it in month four.

Capability sandboxing is why you can let agents act, not just read.

Most enterprises keep agents read-only because the only way to let one act has been to hand it a credential — and a handed-over credential cannot be bounded afterwards. This removes that trade.

● WITHOUT IT

The agent holds a credential

It can reach anything that credential can reach — including things nobody thought about when it was issued.

Blast radius is whatever the credential allows, so the safe answer is always **read-only**.

A prompt-injected or simply mistaken agent has the same reach as a correct one.

Every new action needs a fresh review, because there is no boundary to reason about — only intentions.

The honest trade-off: the WASI toolchain is younger than containers, and not every workload compiles cleanly to it. We use it where the boundary earns its keep — agent tool execution — and ordinary containers everywhere else. This is a pattern, not a migration.

WITH WASI CAPABILITY SANDBOXING

The agent holds only declared doors

It runs inside a sandbox where **only the capabilities it declared exist at all** — no ambient filesystem, network or credential.

Blast radius is the declaration, which is readable, reviewable and diffable in a pull request.

An injected or mistaken agent still cannot reach what was never wired to it.

New actions are a **change to a manifest**, reviewed like code — not a new security engagement.

Net effect: the organisation can say yes to writes, with a bounded answer

What this unlocks commercially: tier-1 and tier-2 write actions ship on the paved road with a manifest review, instead of queueing for a bespoke security assessment. That queue is usually the single largest source of delay between a working agent and a useful one.

Shared vocabulary is what turns compliance and integration from projects into properties.

Two of the most expensive recurring costs in a large enterprise have the same root cause: nothing agrees on what things are called. The operating model fixes that incrementally, as a by-product of registering capabilities.

PAYOFF 01

Agile compliance

When a capability declares what it does, to which entities, under which classification — and the event plane records every use — a control question becomes a **query** rather than a project.

- Controls are evidenced continuously, not reconstructed quarterly
- A new regulation maps to existing declarations instead of a discovery exercise
- Risk reviews shift from interviewing teams to reading the record

Measured as: analyst-days per control cycle, and time to answer a novel

PAYOFF 02

Cross-functional integration

When capabilities are described in business terms rather than endpoints, teams compose them without meeting first. The vocabulary becomes the integration layer.

- An agent asks for a *capability*, not a system — so ownership can change underneath
- Point-to-point integrations stop multiplying with every new consumer
- Teams discover existing capability instead of rebuilding it in a different silo

Sequencing, stated plainly, because this is where these programmes die: we are not starting an ontology programme. In the twelve weeks we do **provenance and grounded retrieval only** — every answer carries its source, timestamp and owner. Vocabulary grows one domain at a time, where it earns its keep, and every capability registered on the paved road contributes to it at no extra cost. If **[CLIENT]** has an ontology or data-mesh programme in flight, this feeds it rather than competing with it.

The first agents build your software. That is deliberate.

Agentic CI/CD and agentic code review are the POC default — internal, reversible, measurable in three weeks, and they exercise every plane of the fabric under genuine load.

1

Trigger

A pull request opens. The event plane starts the work — nobody has to remember to ask.

2

Agentic review

Correctness, tests, security patterns and house conventions — with the reasoning attached to the diff.

3

Test generation

Missing cases proposed as code, in your framework, for a human to accept or reject.

4

Policy gate

Risk tier checked, capability manifests diffed, evidence recorded — automatically.

5

Staged release

Deploy behind the same grants and off switch as any other governed action.

6

Learn

Accept and reject rates feed back. The reviewer improves against your codebase, not a benchmark.

REASON 01

Internal

No customer exposure, no production data, no regulatory surface. Nobody needs to approve an experiment on their own repositories.

REASON 02

Reversible

Every action is a commit, a comment or a pipeline run. The undo procedure already exists and your engineers already know it.

REASON 03

Measurable in three weeks

Review latency, rework rate and change lead time — against a baseline your tooling already holds.

REASON 04

It exercises everything

Identity, policy, grants, triggers, evidence and sandboxing all get used under real load. **The fabric is built using the fabric** — so we feel any friction first, in week four.

An ordinary Tuesday, four months from now.

Priya is a senior engineer on the payments squad. Nothing below is speculative — each moment is a capability the fabric supports at day 90.

08:55

She opens a pull request

Within ninety seconds an agent has reviewed it — correctness, missing test cases, a deprecated internal API she did not know about — with reasoning attached to each line.

TRIGGER · AGENTIC REVIEW

09:40

She disagrees with one comment

She replies, explains why, and merges. The disagreement is recorded and becomes signal — the reviewer gets better against *this* codebase.

HUMAN STAYS IN CHARGE

11:20

She needs customer data for a fix

She asks in the portal. The capability exists, her tier permits it, and she gets a scoped, time-boxed answer with sources cited — no ticket, no waiting.

REGISTRY · GRANTS · RETRIEVAL

14:00

A dependency CVE lands

An agent has already opened the upgrade PR across eleven repositories, run the tests and flagged the two that need a human. She reviews those two.

EVENT TRIGGER · FAN-OUT

16:30

She ships

The policy gate checks the tier and the capability manifest, staged rollout begins, and the evidence record is complete before she closes her laptop.

GOVERNED RELEASE

Now notice who else had a good Tuesday, without attending a single meeting about AI. **Platform** saw every agent action on the same dashboards as everything else. **Risk** had eleven CVE remediations evidenced automatically. **The product owner** got a day of Priya's time back. **Data governance** can show exactly which customer records were read, by whom, under which purpose.

One path. Every agent action, every time.

The value is not the six steps — it is that there is only one route, so there is one thing to learn, one place to look when something is wrong, and one record afterwards.

1

Something starts it

A person asks, a workflow reaches a step, or an event fires on the spine.

2

We know who

The person, the agent and who it is acting for — resolved from your existing identity provider.

3

We find the capability

The registry answers what can do this, who owns it, and whether it is healthy.

4

We check it is allowed

Risk tier, purpose and data class — expressed in business words, not policy syntax.

5

It runs, bounded

Inside a sandbox holding only the doors it declared, with authority that expires.

6

It leaves a record

What happened, who asked, what changed and whether it can be undone.

FOR THE BUSINESS

You ask for an **outcome**, not a system. Ownership can change underneath without breaking you.

FOR ENGINEERING

You inherit identity, policy and telemetry from the template. **You do not write any of step 2, 4 or 6.**

FOR OPERATIONS

One dashboard, one trace, one off switch — per capability, trippable without a deploy.

FOR GOVERNANCE

Step 6 is the audit answer, produced by the system rather than narrated by a person.

Seven roles. Everyone has a lane, nobody needs to become an AI specialist.

Check the middle column. If what you care most about is missing, that is a gap in our understanding and we want it now rather than in week six.

ROLE	WHERE YOU WORK ON THE CANVAS	WHAT YOU OWN	WHAT WE NEED FROM YOU	TIME
Product & business owner	Experience · Outcomes	Naming the capability and the outcome it serves; accepting or rejecting what ships	One named person who can say yes without convening others	2 h/wk
AI practice lead	Agent Fabric	Agent design, the capability backlog, and how quality is evaluated	The capability inventory, and your evaluation approach today	1 d/wk
Architecture	All planes — the coherence owner	That the planes stay separable and the contracts hold as things are added	Current-state view and any standing architectural decisions we must respect	0.5 d/wk
Platform & DevOps	Platform · Event Plane	The paved road, the event spine, and running all of it after week 12	A non-prod namespace in week 1, and authority over the deployment template	1 d/wk
Data engineering	Knowledge · Systems	Capability contracts on data products; freshness and provenance	Catalog read access and the freshness SLA of the sources we ground on	0.5 d/wk
Governance & risk	Control · Knowledge	Risk tiers, what evidence must show, and who may assign a tier	Three tiers roughly defined, plus the one data class that is out of bounds	0.5 d/wk
Our FDE team	Across every plane, then out	Building it, proving it, documenting it and handing it over	Honest correction when we get your environment wrong	5 people

Total client commitment: roughly four days a week across seven people. That is the whole ask. If a role cannot be named today, we need a date — unnamed roles are the single most reliable predictor of a slipped milestone, ahead of any technical risk.

Clear lines. Only five things are genuinely joint.

"Joint" usually means nobody, so we have kept it to the five places where it is genuinely true.

- **We lead** — our team drives, you review
- **You lead** — only your organisation can do this
- **Genuinely joint** — neither of us can do it alone

ACTIVITY	WE LEAD	YOU LEAD	JOINT	NOTE
Choosing the first squad and repositories		■		You know which team wants this most. That matters more than which codebase is tidiest.
Defining risk tiers and who assigns them			■	We bring the shape; only you can set the thresholds.
Environment, identity and repository access		■		The most common cause of slipped milestones. Week 1.
Building the capability registry and grants	■			The two genuinely new components. Handed over as your code.
Event plane conventions on your existing bus			■	Our schema, your operational reality.
Agentic review, test generation, deploy gating	■			Tuned against your codebase and your house conventions.
Deciding what "good enough to merge" means		■		Your engineering standards, not ours. We encode them.
Golden-path templates and documentation	■			Written for your platform, in your conventions.
Value baselines and how we report them			■	Your data, our model. Agreed in week 1, fixed in week 2.
Capability backlog beyond the first workload			■	We facilitate; the business decides.
Evidence review and the control crosswalk			■	Your assessors' frameworks, our architecture evidence.
Running the fabric after week 12		■		Proven in week 11 — a second squad onboards with us out of the room.

Two teams hold the veto. Here is what each one gets.

Platform can quietly refuse to run it. AI practice can quietly route around it. Both concerns are legitimate and both are answered below.

PLATFORM & DEVOPS

You inherit a paved road, not a science project

YOUR WIN

Every agent lands on **your** platform, in **your** pipelines, on **your** dashboards — instead of a shadow stack appearing in a different budget line.

One off switch per capability, trippable by an on-call human without a deploy.

The event plane is a platform asset you would have wanted anyway.

YOUR PART

A non-prod namespace and deploy path — **week 1**.

One engineer at 1 day/week with real authority over the deployment template.

Your current golden-path definition, however informal, so we extend rather than compete.

An honest p99 latency budget — it shapes the design, and we would rather have it early.

You own the road after week twelve. Which is exactly why you get a vote on every decision starting Monday.

AI PRACTICE

This exists so you can say yes more often

YOUR WIN

A capability that fits an approved tier **inherits its policy** instead of negotiating one.

Promotion from pilot to production becomes a checklist, not a committee.

Agent-to-agent delegation becomes expressible — which is what makes multi-agent designs practical rather than theoretical.

Nothing constrains your models, prompts, frameworks or evaluation stack.

YOUR PART

The capability inventory for the first workload.

One practitioner at 1 day/week.

Declaring a capability's scope before first use — about a day of work, once, per capability.

Letting go of the long-lived credential that "just makes the demo work". That pattern closes deliberately.

The commercial argument in one line: tier 1 and tier 2 ship on the road; only tier 3 needs a conversation.

A classification you cannot enforce at access time is a label, not a control.

This is the first place in the estate where the label becomes enforceable — and the last place anyone should start a shadow data programme.

GOVERNANCE & RISK

YOUR WIN

Classification becomes an input to the decision — evaluated at the moment of access, not asserted in a catalog and hoped for.

Purpose becomes expressible.

Authority is issued for a stated purpose; using it for another is refused, and the refusal is itself evidence.

Evidence is continuous. You stop assembling it before reviews and start querying it.

Memory is a governed service.

YOUR PART

Three risk tiers, roughly defined — **week 1**. We will sharpen them with you.

Who has authority to assign a capability to a tier.

The one data class that is categorically out of bounds for this POC.

Which assessment frameworks bind you, so the week-9 crosswalk targets the right ones.

0.5 day/week.

DATA ENGINEERING

YOUR WIN

No new pipelines in the twelve weeks. Curated existing sources only. If it needs an ingestion project, it is out of scope by definition.

Retrieval is never asked to be a security boundary. Access filtering happens in the control plane, so your layer stays a data layer.

Data products gain a capability contract — what it answers, freshness, owner, classification — reusing your catalog rather than replacing it.

YOUR PART

Read access to the catalog — **week 1**.

The freshness SLA of the sources the first workload grounds on.

One engineer at 0.5 day/week.

An honest answer on which sources are trustworthy enough to ground on today, and which are not.

If **[CLIENT]** has an ontology, data-mesh or catalog programme in flight: we feed it rather than compete with it. Everything the fabric emits — capability semantics, provenance, ownership, purpose — is structured input that programme can consume. We are not waiting for it and we are not duplicating it.

Five segments, three levels. The twelve weeks delivers level one across all five.

Nobody needs to be at level three. Everything to the right of the blue column is a choice made later, with better information than anyone in this room has today.

	LEVEL 1 · FOUNDATION — WEEKS 1-12	LEVEL 2 · SCALE — QUARTERS 2-3	LEVEL 3 · COMPOUNDING — QUARTER 4+
Agent fabric Registry, orchestration, delegation	POC Capabilities registered with owner, tier and health. One squad building on templates.	Self-service onboarding across squads. Delegation between agents. A capability backlog owned by the business.	TARGET Portfolio management — capabilities retired, versioned and reused as deliberately as microservices.
Event plane Observability and triggers	POC One correlated context across agents and services. Events trigger the delivery agents.	Business events trigger business capabilities. Evidence stream feeds risk reporting directly.	TARGET Governed autonomous operations — the system reacts within policy, and humans supervise by exception.
Knowledge Meaning, grounding, memory	POC Provenance and grounded retrieval. Every answer carries source, timestamp and owner.	Shared vocabulary for one or two domains. Entity resolution where it pays. Reusable knowledge products.	TARGET Federated ontology across domains — compliance and integration become properties, not projects.
Safe execution Tiers, sandboxing, authority	POC Three risk tiers. Capability manifests reviewed as code. Sandboxed writes on tier 1 and 2.	Tier assignment self-served against clear criteria. Security review becomes exception handling.	TARGET Policy as code at scale — new regulation lands as a policy change, not a programme.
Agentic delivery How software gets built	POC Review, test generation and deploy gating on one squad's repositories.	All squads. Cross-repository maintenance — dependency, CVE and migration work handled by fan-out.	TARGET Agents hold routine maintenance entirely; engineers spend their time on design and judgement.

The question we would like answered today: which of these five matters most to **[CLIENT]**? That is the one we push hardest on inside the twelve weeks — and different people in this room will pick differently, which is worth surfacing now rather than in month three.

Six moves. Each with a reason, its payoffs, and an honest cost.

A roadmap without trade-offs is a wish list. The test we apply: **if an investment only pays off once, it is a feature — not a platform move.**

NOW · IN THE 12 WEEKS

Unify the event plane

Observability and eventing are the same spine funded from two budgets. Build it once.

- Extensible observability across every agent and service
- Agent triggers — work starts from events, not reminders
- Evidence produced as exhaust

COSTS YOU

Schema discipline, back-pressure handling and a named owner. One spine means one thing to get right.

NOW · IN THE 12 WEEKS

Sandbox agent execution

The only reason agents stay read-only is that a handed-over credential cannot be bounded afterwards.

- Write actions become approvable at tier 1 and 2
- Manifest review replaces the bespoke security queue
- Blast radius becomes a diff, not a discussion

COSTS YOU

A younger toolchain than containers, and not every workload compiles to it. Use it where the boundary earns its keep.

QUARTER 2 · CHOICE

Agentic delivery everywhere

The first squad's throughput gain is the proof; the portfolio gain needs every squad on it.

- Cycle-time improvement across the whole engineering org
- Consistency — house conventions applied uniformly
- Cross-repo maintenance by fan-out, not by ticket

COSTS YOU

Review fatigue is real. Needs quality gates and a way for engineers to reject and be heard, or trust erodes fast.

QUARTER 2-3 · CHOICE

Shared vocabulary for one domain

Compliance effort and integration cost have the same root cause: nothing agrees on what things are called.

- Agile compliance — controls evidenced, not reconstructed
- Cross-functional integration without a meeting first
- Capability reuse rather than silo rebuild

COSTS YOU

Needs a domain owner with authority, and shows value slowly. Start with one domain — never enterprise-wide.

QUARTER 3 · CHOICE

Policy as code at scale

Tier assignment done by humans is a queue; done by criteria it is a checklist.

- Squads self-serve tiers against clear criteria
- Security review becomes exception handling
- New regulation lands as a policy change, not a programme

COSTS YOU

Policy authoring becomes a real skill someone must hold. Done badly, it is worse than the queue it replaced.

QUARTER 4+ · WHERE IT GOES

Governed autonomous operations

Once evidence is trustworthy, the system can respond within policy and humans supervise by exception.

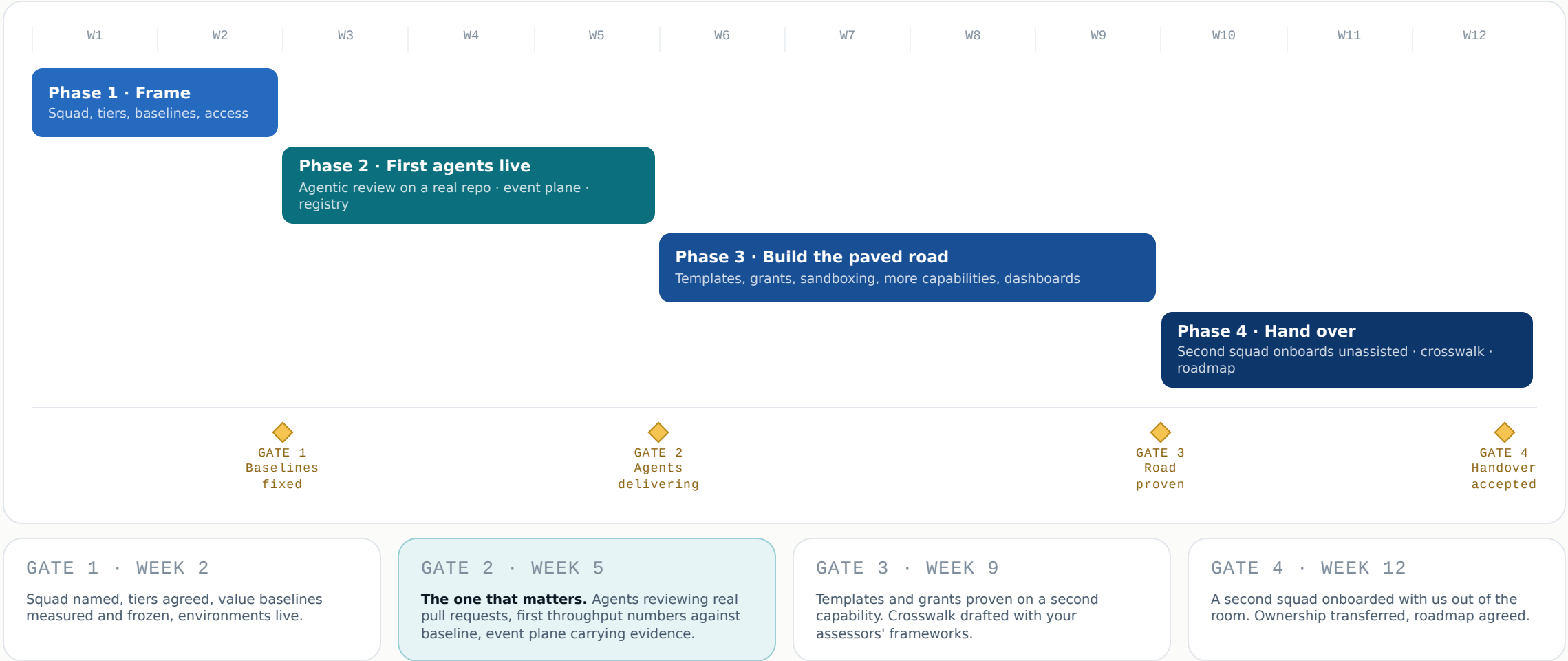
- Remediation and scaling handled inside guardrails
- Operational cost decoupled from portfolio growth
- Engineers on design and judgement, not toil

COSTS YOU

Requires evidence maturity you will not have for a while. **On this page because it is where this goes — not because we think you should do it next year.**

Agents reviewing real pull requests in week three.

Four phases, four one-page gates, circulated 24 hours ahead. Starting [START DATE].



Every item is a noun you can point at.

No workstreams, no enablement, no alignment. If it cannot be opened, run or read, it is not on this slide.

DAY 1 · END OF WEEK 1

The **first squad** named, with a product owner

Three **risk tiers** defined, with an assignment owner

Value **baselines measured** — review latency, rework, lead time

Seven **named people** and a fixed weekly slot

A **decision log** with entries in it

Environment and repository access **requested, with dates**

DAY 30 · END OF WEEK 4

Agents reviewing real pull requests on your repositories

A **capability registry** with owners, tiers and health

The **event plane** carrying one correlated context

First **throughput numbers** against your own baseline

Sandboxed execution running **tier-1 write actions**

A frank week-5 gate paper, **including what is not working**

DAY 90 · END OF WEEK 12

A **paved road**: templates, registration, policy inheritance, tests, dashboards, off switches

Agentic delivery across **review, tests, dependencies, release notes and deploy gating**

A **capability portal** showing what exists, who owns it and whether it is healthy

A **control crosswalk** with sample evidence, reviewed by your assessors

A **second squad** that onboarded from documentation alone

A **measured value report** and an agreed next-quarter roadmap

All of it yours whether or not you continue with us.

Three things, and the dates that matter.

If access lands in week three instead of week one, gate 2 moves to week seven. That is not a complaint — it is arithmetic, and it is the most common way this plan slips.

ACCESS

Repositories for the first squad — read and PR	W1
CI/CD pipeline access for that squad	W1
Non-production namespace and deploy path	W1
Identity provider tenant — non-prod, with admin	W1
Observability backend — write and query	W2
Service catalog / CMDB — read	W2

The honest question: how long do these genuinely take here — not the SLA, the reality? If it is six weeks, it starts before the contract is signed.

DECISIONS

First squad and product owner confirmed	Today
Agentic delivery confirmed as first workload	Today
Three risk tiers, and who assigns them	W1
Incumbent platforms confirmed	W1
Data class that is out of bounds	W1
Value baselines measured and frozen	W2
What "good enough to merge" means here	W2

PEOPLE & RHYTHM

Seven named roles — slide 14	Today
Weekly 45-minute working session, fixed slot	W1
Gate reviews — 60 min, sponsor present	W2/5/9/12
Shared channel and one written decision log	W1
Your change-freeze calendar	W1

One question we need answered today: what falls inside this twelve-week window — freezes, audits, quarter close, holidays? Those have derailed more of these than any technical risk.

Five decisions, and we start Monday.

Taken one at a time. Where we cannot decide today, we need a name and a date — an undecided item with no owner becomes a slipped gate in week three.

DECISION	WHAT WE NEED TO HEAR	OWNER	STATUS
1 The first squad	A named team, their repositories, and a product owner who can say yes without convening others. Pick the team that <i>wants</i> this, not the tidiest codebase.	—	OPEN
2 Agentic delivery as the first workload	Confirmation that we start with code review, test generation and deploy gating — and that business capabilities are path two, designed in week ten.	—	OPEN
3 The incumbent platforms	Which identity provider, observability backend, gateway and event bus we build on rather than replace.	—	OPEN
4 The seven roles and the rhythm	Names against slide 14 — or a date by which we will have them — and a fixed weekly slot starting this week.	—	OPEN
5 The value measures	Which of the four drivers we report against, and who owns the baseline. We measure what you will actually be asked about.	—	OPEN

WITHIN 24 HOURS

Written confirmation of every decision above, access requests raised with dates, and the first weekly session in calendars. Nothing agreed in this room survives on memory alone.

WITHIN 5 WORKING DAYS

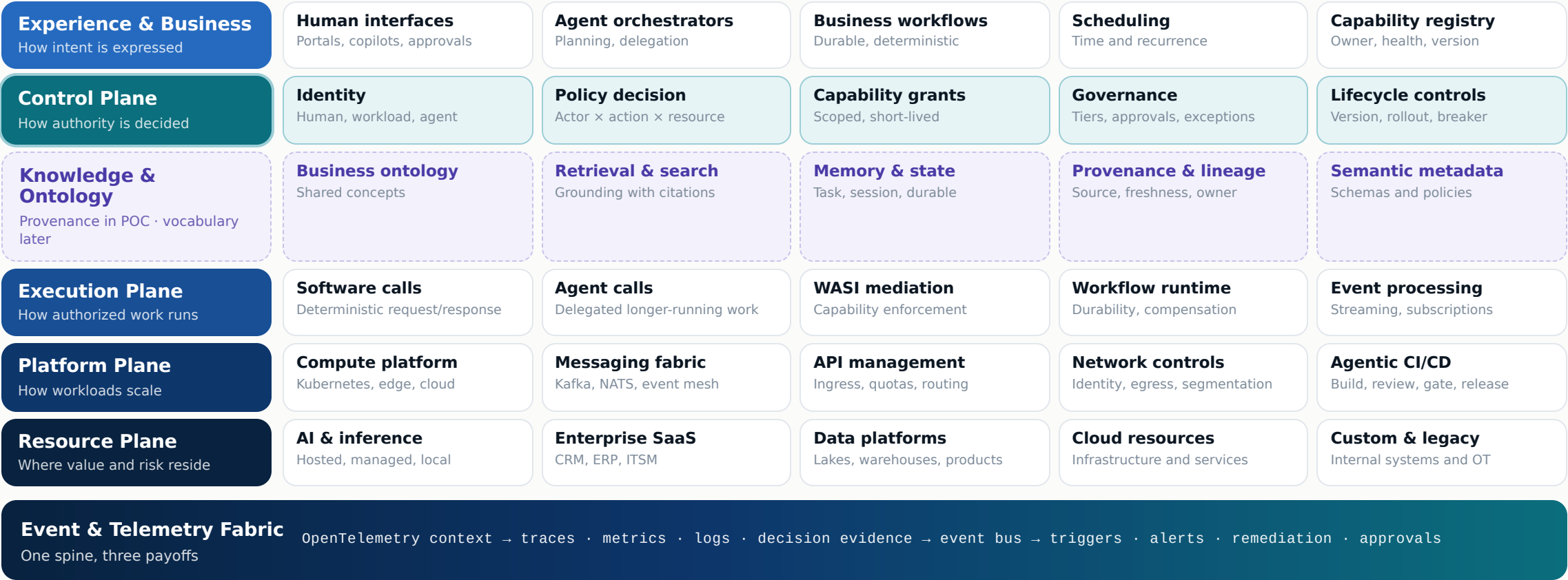
The week-1 working session — half a day, the seven named people, baselines measured, and the estate map on slide 7 completed properly rather than live-edited.



**Build the fabric once.
Let every team use it.
Start with the software
you already know how to judge.**

The canvas in architectural form.

Responsibilities are fixed; product selections are contextual. This is slide 4 for an audience that reads architecture faster than infographics.



Five responsibilities, four of which you already have.

Standardize these; keep every product underneath them replaceable. The one that is usually genuinely new is capability grants.

RESPONSIBILITY	ANSWERS	IN ESTATE?
Identity Human, workload, agent	Who or what is acting, and on whose behalf?	 REUSE
Policy decision Actor × action × resource × context	May this actor perform this action, right now?	 EXTEND
Capability grants Scoped, short-lived, attenuable	How is that decision carried to the thing enforcing it?	 NEW
Governance Tiers, approvals, exceptions	Who decided this action class was acceptable, and when?	 REUSE
Lifecycle controls Version, rollout, breaker	How do we stop it — in seconds, without a deploy?	 EXTEND

A GRANT IS NOT A ROLE

A role describes what an actor **is**, and persists. A grant describes what an actor **may do** — bound to audience, action, scope and expiry — and is designed to die.

- **Audience-bound** — usable only at the intended resource
- **Attenuable** — a holder can narrow it, never widen it
- **Short-lived** — expiry is the default revocation mechanism
- **Verifiable offline** — routine work needs no round trip

The design tension, resolved in week 4: offline verification gives you latency; online decisions give you instant revocation. High-risk actions get the round trip; routine actions get local verification with a short TTL. Where **[CLIENT]** draws that line is a governance decision, not an engineering one.

Eight steps, and the record they produce.

Implementation varies by risk tier; the logical sequence does not. Steps 4 and 5 are the crux — the decision, and the conveyance of that decision.



```
trace_id 4f2a...c81b   on_behalf_of user: [EMPLOYEE]   actor agent:pr-reviewer/v3.1   workload spiffe:///.../review
capability repo.pr.comment   decision permit   policy tier1-delivery@v11   grant aud=scm scope=pr:comment ttl=90s
enforced_at wasi-sandbox/v2.4   manifest net:scm-api · fs:none · env:none   side_effects 1 comment · reversible   outcome ok · 214ms
```

This record is the assurance deliverable. If an auditor can read it without a human narrating what the agent "was trying to do", every claim on the crosswalk becomes checkable rather than asserted.

Five invariants, each with a test we run in week nine.

Principles are cheap; principles with a stated test are an engineering commitment. Your security and SRE teams are invited to design the adversarial cases.

INVARIANT	WHAT IT MEANS	HOW YOU TEST IT — AND WHAT A PASS LOOKS LIKE
1 Authority is explicit	No ambient access. Every action attributable to an identity and a decision.	Pick any action from the last 24 hours at random. Produce actor, human principal, decision, grant and expiry in under 60 seconds , without asking an engineer.
2 Delegation only narrows	Sub-agents receive subsets of authority — never amplification.	Have a sub-agent attempt an action its parent could not perform. It must fail closed , and the denial must appear in evidence with the attenuation reason.
3 Execution is bounded	The capability manifest is the blast radius, and it is reviewable.	Attempt an action outside the declared manifest — a network egress or file read never wired in. It must be impossible, not merely refused .
4 Everything is observable	Policy, routing, runtime and business outcome share one correlated context.	One trace identifier joins intent → decision → grant → runtime → backend → outcome across at least three systems , with no manual stitching.
5 Technology is replaceable	Interfaces and responsibilities standardized; products chosen by context.	Swap the policy engine — or the gateway — behind the same contract. Zero service code changes , and evidence continuity preserved across the swap.

Open invitation: your security and SRE teams design the adversarial cases for tests 2, 3 and 5. We would rather be broken by you in week nine than by an incident in month nine. If you want a sixth invariant, we will add it.

Isolation cost matched to consequence, not applied uniformly.

Tiers are set with [CLIENT] in week one. These are the shapes we start the conversation from — not a standard we impose.

TIER	CHARACTERISTIC	EXAMPLES IN DELIVERY	ISOLATION	APPROVAL PATH
Tier 1 SELF-SERVICE	Reversible, internal, no personal or regulated data, bounded output	PR comments, test generation, release notes, documentation	WASI sandbox, declared manifest, short-lived grant	Manifest reviewed in the pull request. No separate approval.
Tier 2 OWNER APPROVES	Reversible writes to systems of record, or reads of classified data	Dependency upgrades, branch creation, ticket updates, grounded answers on customer data	WASI sandbox plus online policy decision for the write	Capability owner approves the tier assignment once, not each use.
Tier 3 GOVERNED	Irreversible, customer-facing, financially material, or regulated action	Production deploys to regulated systems, anything touching money or customers directly	Sandbox, online decision, human in the loop, staged rollout	Named human approval per action or per batch, recorded as evidence.

WHY NOT ISOLATE EVERYTHING AT TIER 3

Uniform maximum isolation is either unaffordable everywhere or quietly bypassed where it hurts. Tiering is what keeps the strong controls credible in the places they actually matter.

WHERE THE POC SITS

Agentic delivery is almost entirely **tier 1**, with a small tier-2 surface for dependency upgrades and branch writes. That is precisely why it is the right first workload — real writes, bounded consequence.

Evidence in a shape your existing frameworks already accept.

Which of these bind [CLIENT]? Mark them and we scope the week-9 crosswalk accordingly.

ARCHITECTURE EVIDENCE	NIST AI RMF	ZERO TRUST · SP 800-207	ISO/IEC 42001	SOC 2 · EU AI ACT
Attribution record Actor, human principal, decision	MAP · MANAGE	Policy decision point logging	Operational planning & control	CC6 logical access · Art. 12 record-keeping
Grant scope & expiry Audience, action, TTL, attenuation	GOVERN · MANAGE	Per-session least privilege	Resource & access control	CC6 logical access
Capability manifest Declared, reviewed, diffed	MAP · MEASURE	Asset & resource classification	AI system impact assessment	CC6 · Art. 9 risk management
Denial evidence Refused attempts and reasons	MEASURE	Continuous verification	Performance evaluation	CC7 system operations
Correlated invocation trace Intent → outcome, ≥3 systems	MEASURE · MANAGE	Monitoring & diagnostics	Monitoring, measurement, analysis	CC7 · Art. 12 logging
Circuit breaker & revocation Time-to-stop, measured	MANAGE	Dynamic policy enforcement	Incident & nonconformity	CC7 · Art. 14 human oversight

Stated plainly: a twelve-week proof of concept produces *evidence samples* mapped to these controls. It does not produce a certified control, an audit pass or a compliance assertion. What it produces is a defensible case that the evidence a full implementation generates is the evidence your assessors need.

What could go wrong, and when we would tell you to stop.

Almost every mitigation below is early detection rather than clever engineering. We are happy to write the stop conditions into the statement of work.

RISKS

	RISK	MITIGATION	OWNER
1	Access slips beyond week 2	Requests raised day 1, tracked in the decision log, escalated weekly. Gate 1 is not complete without it.	Sponsor
2	The first squad does not want it	Pick the team that asked, not the one assigned. A named fallback squad costs nothing before week 3.	Joint
3	Agent review quality is poor on this codebase	Measured from week 3 with accept/reject rates. Tuning is expected; if it is still poor at gate 2, we say so.	Us
4	Review fatigue erodes trust	Signal-to-noise tracked explicitly. Agents stay quiet by default and earn the right to comment.	Joint
5	A change freeze falls in the window	Freeze calendar reviewed week 1; phases shifted around it rather than compressed into it.	Platform
6	Key people pulled to an incident	Named deputy per role. A written decision log means context survives an absence.	Sponsor
7	Scope widens after a good demo	Every addition displaces something of equal size, recorded in the gate note with both names on it.	Joint

STOP CONDITIONS

- 1 • **No measurable throughput signal by gate 2.** If agentic delivery cannot move review latency or rework on a willing squad in five weeks, the workload choice is wrong.

2 • **Enforcement cost is unaffordable.** If policy and sandboxing add more than the agreed budget to the hot path and cannot be tiered or cached down, the model does not fit this estate.

3 • **Evidence cannot be correlated.** If a trace cannot be joined across three systems without manual stitching, every downstream assurance claim rests on sand.

4 • **The squad will not adopt without us in the room.** This is the failure mode that kills programmes six months after the consultants leave.

5 • **[YOUR FIFTH]** — proposed by the client. We will write it in.

How a stop actually works: a written finding within 48 hours, joint review inside a week, then one of three outcomes — remediate with a named change and a new date, re-scope, or stop. **You keep the artefacts in all three cases.**

Choose technology by responsibility, not by category.

We hold no commercial relationship with anything named here. If your incumbent is missing, that is a gap in our knowledge rather than a judgement on your choice.

CONTROL & IDENTITY

RESPONSIBILITY	REPRESENTATIVE OPTIONS	WORKING POSTURE
Enterprise identity	Entra, Okta, Keycloak, cloud IAM	REUSE FIRST No parallel identity estate for agents.
Workload identity	SPIFFE/SPIRE, mesh identity, cloud workload identity	ADD WHERE NEEDED Where service accounts are too blunt.
Policy decision	OPA/Rego, Cedar, OpenFGA, IAM-native ABAC	SMALL VOCABULARY Business concepts, not engine syntax.
Authority conveyance	Signed tokens, caveated credentials, mTLS-bound grants	HYBRID Local for routine; online for high risk.

EXECUTION, PLATFORM & EVIDENCE

RESPONSIBILITY	REPRESENTATIVE OPTIONS	WORKING POSTURE
Agent isolation	WASI runtimes, sandboxed containers, microVMs	RISK TIERED Match isolation cost to consequence.
Event fabric	Kafka, NATS, Pulsar, cloud event services	REUSE FIRST One spine; conventions, not new infrastructure.
Telemetry contract	OpenTelemetry APIs, SDKs, collectors, conventions	STANDARDIZE One invocation context, end to end.
Observability backend	Grafana, Datadog, Splunk, Elastic, cloud APM, SIEM	REUSE FIRST No separate estate solely for agents.

Selection criteria applied throughout: explainability of decisions · deterministic evaluation · attenuation and audience binding · offline verifiability · blast-radius containment · operational maturity in *your* hands, not ours.

Six terms this deck uses precisely.

Each is commonly used loosely. Where a definitional argument starts, it usually ends here.

Agent fabric

The shared substrate that makes the second, tenth and hundredth agent cheap to build and safe to run: registry, orchestration, delegation, identity, policy, event spine and paved road. Distinct from any individual agent.

Capability

A named, owned, tiered thing an agent or service can do, described in business terms rather than as an endpoint. The unit of registration, of policy and of reuse.

Capability grant

A short-lived, audience-bound, attenuable credential conveying a specific authorization decision to the point that enforces it. Distinct from a role, which describes what an actor is and persists indefinitely.

Capability manifest

The declaration of exactly which doors an agent's sandbox opens — network, filesystem, credentials. Reviewed and diffed like code; it *is* the blast radius.

Event plane

One spine carrying both telemetry and triggers, so observability, agent activation and evidence come from a single investment rather than three.

Paved road

The supported, templated, instrumented route by which a team registers and operates a capability. Its quality is measured by adoption without its authors present.

Week-0 intake — the fields this deck is parameterised on.

Every bracketed field resolves from this sheet. Press **E** on any slide to highlight them.

ENGAGEMENT

[CLIENT]	—
[DATE] · kickoff	—
[START DATE] · week 1	—
[ENGAGEMENT ID]	—
Binding assessment frameworks	—
Change freezes in window	—
Executive sponsor	—

FIRST SQUAD & MEASURES

Squad name and product owner	—
Repositories in scope	—
Fallback squad	—
Review latency baseline	—
Rework rate baseline	—
Change lead time baseline	—
[\$X] incremental run cost ceiling	—
Which value drivers we report	—

ESTATE & PEOPLE

Identity provider	—
CI/CD platform	—
Observability backend	—
Event bus / messaging	—
Compute platform	—
Service catalog / CMDB	—
Data class out of bounds	—
Seven named roles · slide 14	—

Where this arrives filled in before the session, the kickoff runs thirty minutes shorter and slide 7 becomes a review rather than a live exercise. That is the single highest-leverage piece of preparation available.